



## TASK

# Capstone Project – Build Your Developer Portfolio

[Visit our website](#)

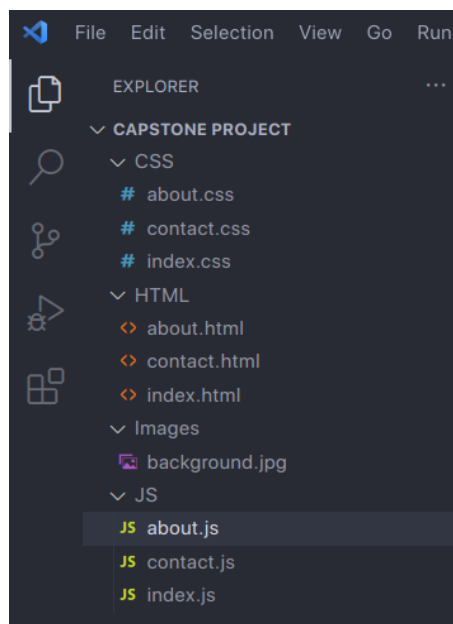
# Introduction

In this task, you will consolidate the knowledge you have learnt and apply it to a real-world application. You'll be tasked with a set of criteria to meet, and the rest is up to you! Remember, it is worth putting some extra time and effort into this project as it can become part of your developer portfolio.

## BEFORE YOU BEGIN

A key focus of this project will be ensuring that your JavaScript, CSS, and HTML are correct, well-formatted, and readable. In this regard, make sure that you do the following before submitting your work:

1. Ensure that you use a good directory structure to organise your web application. Keep your entire project in a directory with a meaningful name. Within this directory, make sure you have separate directories for any images, CSS, and JavaScript you use. Below is an example of what an organised folder structure would look like:



2. You can also use tools like [Chrome's developer console](#) to troubleshoot your code.
3. Make sure to refactor your code and include comments throughout your JavaScript, HTML, and CSS code. During the process, ask yourself the following questions:

- Are you using functions to improve the modularity and readability of your code?
- Have you given your variables and functions meaningful names that adhere to the [Airbnb JavaScript Style Guide](#) or [Google JavaScript Style Guide](#) naming conventions?



## Capstone project

Build a JavaScript application that interacts with [TheMealDB](#) API to take and complete orders of the “chef’s favourite meals”. Ensure to implement user interactions using DOM manipulation and event handlers. The application should also utilise the Fetch API for consuming TheMealDB API, and **sessionStorage** to store order details.

You will be utilising the API method to “[filter by main ingredient](#)”. Explore the list of available [ingredients](#) and observe that the “**strIngredient**” encompasses items such as pork, lemon, mint, and more. Visit the “[filter by main ingredient](#)” URL, then substitute “chicken\_breast” in the URL with “beef” and send the request. The API will return a JSON response with recipes that match the ingredient. Try “Fanta” to see how the API returns null values when no results are found.

1. Taking orders:
  - Allow users to enter the main ingredient.
  - Call the API to retrieve a list of the chef’s favourite meals based on the main ingredient provided.
  - Make use of the **random()** function to randomly select a chef’s favourite meal and set it as the order.
  - The order will be defined by its description, order number, and completion status, which can either be completed or incomplete.
  - The main ingredient is only used to retrieve the description, i.e., the **strMeal** field, from the API, and needn’t be stored as part of the order details.

- When the user enters the main ingredient, convert characters to lowercase and replace spaces with underscores for TheMealDB API. For example, "Garlic Powder" becomes "garlic\_powder". The resulting URL will be:

[https://www.themealdb.com/api/json/v1/1/filter.php?i=garlic\\_powder](https://www.themealdb.com/api/json/v1/1/filter.php?i=garlic_powder)

- If the API returns null, provide a suitable response, and ask the user for another ingredient. A URL example that would return a `{"meals":null}` value is:

<https://www.themealdb.com/api/json/v1/1/filter.php?i=fanta>

**Tip:** You may find it useful to make use of [recursion](#) in this case.

## 2. Storing orders:

- Store the order details in **sessionStorage**.
- Ensure that each new order has a unique order number.
- Make sure that your session storage contains the last generated order number (which is the number of items in storage session + 1), to avoid having to loop over all the orders when storing them.

## 3. Displaying and completing orders:

- Display all the incomplete orders stored in session storage. Present the order number and description only.
- Allow users to enter the order number to mark as complete, or enter zero for not completing an order.
- Once an order's completion status is updated, the status should be committed to the session storage.
- Display an appropriate response message if the order number doesn't exist.
- **Hint:** You can use the **filter()** function to select only incomplete orders.

## 4. Storage format:

- Store the collection of orders as a single value in a JSON array.
- The last order number should be stored as a separate value.
- **Hint:** You can retrieve and return values from **sessionStorage** using the **getItem()** method.

## 5. HTML and CSS:

Use HTML to structure your application and apply CSS for styling to make it visually appealing. You have the freedom to choose how to organise the

layout based on your design preferences. However, ensure that the application is responsive, adjusting its layout for different screen sizes using techniques like media queries. The goal is to create a functional and accessible application, but how you choose to design it is up to you.

**Important:** Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



## Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.

---